

Theoretical Assignment 9 – PySpark

Complete notes covering UDFs, window functions, explode/collect_list, multiline JSON, API ingestion, XML rowTag, Spark architecture, cache/persist, DataFrame cleaning, and bucketing.

1. What are User-Defined Functions (UDFs) in PySpark?

UDF stands for User-Defined Function. A UDF allows a developer's Python function to be registered and used on PySpark DataFrame columns. UDFs are useful when the required custom logic is not conveniently available through Spark's built-in functions.

```
Python function
  ↓
Register as UDF
  ↓
Apply to DataFrame column
  ↓
Result becomes a DataFrame column
```

Example

```
data = [
    ("Abhishek", 50000),
    ("Rahul", 70000),
    ("Ananya", 90000)
]

df = spark.createDataFrame(
    data, ["name", "salary"]
)

def salary_category(salary):
    if salary < 60000:
        return "Low"
    elif salary < 80000:
        return "Medium"
    else:
        return "High"

from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

salary_category_udf = udf(
    salary_category, StringType()
)

result = df.withColumn(
    "salary_category",
    salary_category_udf("salary")
)

result.show()
```

name	salary	salary_category
Abhishek	50000	Low
Rahul	70000	Medium
Ananya	90000	High

UDF vs Normal Python Function

```
Normal Python function
  ↓
Works with normal Python values

PySpark UDF
  ↓
Python function registered with Spark
  ↓
Can be used with DataFrame columns
```

Your previously studied flatten_df() example

The nested schema you studied contains StructType fields such as image and thumbnail. The `flatten_df()` code is **not a UDF**. It is a normal Python helper function that recursively examines the DataFrame schema and builds Spark column expressions.

```
root
|-- id: string
|-- image: struct
|   |-- height: long
|   |-- url: string
|   |-- width: long
|-- name: string
|-- thumbnail: struct
|   |-- height: long
|   |-- url: string
|   |-- width: long
|-- type: string

def flatten_df(df):

    def get_columns(schema, prefix=""):
        columns = []

        for field in schema.fields:
            column_name = (
                f"{prefix}.{field.name}"
                if prefix else field.name
            )

            if type(field.dataType) == StructType:
                columns.extend(
                    get_columns(
                        field.dataType, column_name
                    )
                )
            else:
                alias_name = column_name.replace(".", "_")
                columns.append(
                    df[column_name].alias(alias_name)
                )

        return columns

    return df.select(*get_columns(df.schema))
```

```
image
    height
    url
    width
```

becomes:

```
image_height
image_url
image_width
```

```
and similarly:
thumbnail_height
thumbnail_url
thumbnail_width
```

Important distinction: UDF = custom function registered for DataFrame column processing. `flatten_df()` = normal Python function that constructs Spark column expressions from a nested schema.

2. What are Window Functions in PySpark?

A window function performs a calculation across related rows while keeping the original rows. This differs from `groupBy()`, which aggregates rows into fewer rows.

```
groupBy()
Rows → aggregated rows
```

```
Window function
Rows → same rows + calculated value
```

Window specification

```
Window.partitionBy(...).orderBy(...)
```

partitionBy() defines the group within which the calculation happens. **orderBy()** defines the order inside the window. For the ranking/position functions in this assignment, orderBy is required and partitionBy is optional. Without partitionBy, the entire DataFrame is one window.

dense_rank()

Revenue	dense_rank
1000	1
900	2
900	2
800	3

rank()

Revenue	rank
1000	1
900	2
900	2
800	4

row_number()

Revenue	row_number
1000	1
900	2
900	3
800	4

lag()

Returns the value from a previous row.

Month	Revenue	Previous Revenue
Jan	100	NULL
Feb	150	100
Mar	120	150

lead()

Returns the value from the next row.

Month	Revenue	Next Revenue
Jan	100	150
Feb	150	120
Mar	120	NULL

ntile()

Divides ordered rows into n approximately equal buckets.

```
ntile(4)
```

```
Rows: 1 2 3 4 5 6 7 8
```

```
Bucket 1 → 1 2
```

```
Bucket 2 → 3 4
```

```
Bucket 3 → 5 6
```

```
Bucket 4 → 7 8
```

Monthly Revenue Example

```
month_revenue_df = gross_df.withColumn(
    'year_month',
    date_format('order_date', 'yyyy-MM')
)

total_revenue_df = (
    month_revenue_df
    .groupBy('year_month')
    .agg(
        sum('gross_amount').alias("total_revenue")
    )
)

window_spec = Window.orderBy(
    desc('total_revenue')
)

total_revenue_df1 = (
    total_revenue_df
```

```

.withColumn(
    'dense_rank_res',
    dense_rank().over(window_spec)
)
.withColumn(
    'rank_res',
    rank().over(window_spec)
)
.withColumn(
    'rn_res',
    row_number().over(window_spec)
)
)

```

Example:

Month	Revenue	Dense Rank	Rank	Row Number
Feb	80000	1	1	1
Mar	80000	1	1	2
Jan	50000	2	3	3
Apr	30000	3	4	4

For a top-3 question, dense_rank <= 3 means the top three distinct rank levels; row_number <= 3 means exactly three rows.

3. What are explode() and collect_list() in PySpark?

explode() converts array elements into separate rows. collect_list() collects multiple row values into an array.

```

explode()
Array → multiple rows

```

```

collect_list()
Multiple rows → array

```

```

data = [
    (1, ['sales', 'finance', 'HR']),
    (2, ['sales', 'HR']),
    (3, ['sales', 'security'])
]

```

```

df = spark.createDataFrame(
    data, ['id', 'dept']
)

```

```

df.show()
df.printSchema()

```

explode()

```

df1 = df.withColumn(
    'dept', explode('dept')
)

```

```

df1.show()
df1.printSchema()

```

```

+---+-----+
|id|dept|
+---+-----+
|1|sales|
|1|finance|
|1|HR|
|2|sales|
|2|HR|
|3|sales|
|3|security|
+---+-----+

```

collect_list()

```

df2 = (
    df1
    .groupBy("id")
    .agg(
        collect_list('dept').alias('dept')
    )
)

```

```

df2.show()
df2.printSchema()

```

```

+---+-----+

```

```

id |dept      |
+---+-----+
1  |[sales, finance, HR] |
2  |[sales, HR]         |
3  |[sales, security]   |
+---+-----+

```

`collect_list()` does not remove duplicates. For unique values, `collect_set()` is the related function.

4. What does `multiLine=True` mean when reading JSON?

```

donut_df = spark.read.json(
    'donut.json',
    multiLine=True
)

```

`multiLine=True` tells Spark that a JSON document/record may span multiple physical lines.

```

{
  "id": 1,
  "name": "Chocolate Donut",
  "price": 100
}

```

The single JSON document occupies multiple lines.
`multiLine=True` → Spark reads the complete document.

JSON Lines/JSONL commonly stores one JSON object per line. Multi-line JSON is different because one JSON document can span multiple lines.

5. How can you retrieve data from an API response and load it into a PySpark DataFrame?

```

import requests
import json

url = 'https://randomuser.me/api/0.8/?results=1'

response = requests.get(url)
json_data = response.json()

print(type(json_data))

with open('url_data.json', 'w') as f:
    json.dump(json_data, f, indent=4)

df = spark.read.json(
    'url_data.json',
    multiLine=True
)

```

```

df.printSchema()
df.show()

```

```

API
↓
requests.get()
↓
HTTP response
↓
response.json()
↓
Python object
↓
Save JSON
↓
spark.read.json()
↓
PySpark DataFrame

```

`requests` communicates with the API. Spark then reads and processes the resulting JSON.

6. How do you read XML files in PySpark? What is `rowTag`?

```

book_df = spark.read.xml(
    'new_book.xml',
    rowTag='catalog'
)

```

)

rowTag tells the XML reader which XML element should be treated as one DataFrame row.

```
<books>
  <catalog>
    <id>1</id>
    <name>Python</name>
    <price>500</price>
  </catalog>
  <catalog>
    <id>2</id>
    <name>PySpark</name>
    <price>700</price>
  </catalog>
</books>
```

```
rowTag='catalog'
```

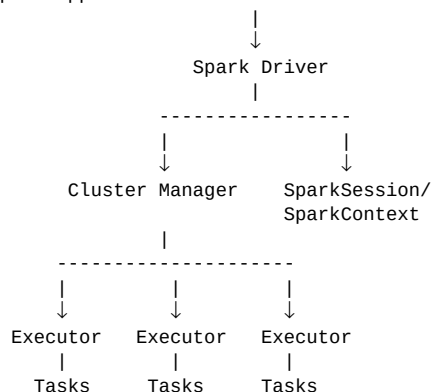
```
catalog 1 → Row 1
```

```
catalog 2 → Row 2
```

This is especially important for nested XML containing repeating record elements. The XML datasource must be available in the Spark environment.

7. Explain the Architecture of Apache Spark and PySpark

PySpark Application



Driver

The Driver is the main coordinating process. It runs application code, creates the Spark session/context, builds execution plans, coordinates executors, and tracks jobs and tasks.

Cluster Manager

The Cluster Manager manages cluster resources and helps Spark obtain resources for executors. Common choices include Standalone, YARN, and Kubernetes.

Executors

Executors run on worker machines. They execute tasks, process partitions, can store cached/persisted data, and report status/results to the driver.

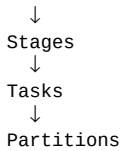
Partitions

Spark divides data into partitions so pieces of data can be processed in parallel.

```
100 GB data
↓
Partition 1
Partition 2
Partition 3
...
Partition N
```

Jobs, Stages and Tasks

```
Action
↓
Job
```



An action triggers a job. Spark divides the job into stages, and tasks execute the work for partitions.

8. What is the difference between `cache()` and `persist()`?

Both are used to keep computed data available for reuse so Spark can avoid recomputing the same lineage repeatedly.

```

cache()
↓
Use the default storage level

persist()
↓
Choose a storage level

```

`cache()`

```
df.cache()
```

`cache()` is convenient when the default caching storage level is sufficient.

`persist()`

```

from pyspark import StorageLevel

df.persist(
    StorageLevel.MEMORY_AND_DISK
)

```

`persist()` allows an explicit storage level to be selected.

Storage Levels

```

MEMORY_ONLY
Stores data in memory. If it does not fit,
uncached partitions may be recomputed.

MEMORY_AND_DISK
Stores data in memory and spills excess
partitions to disk.

DISK_ONLY
Stores persisted data only on disk.

MEMORY_ONLY_SER / MEMORY_AND_DISK_SER
Serialized storage can reduce memory usage,
at the cost of serialization/deserialization CPU.

MEMORY_ONLY_2 / MEMORY_AND_DISK_2
Replication factor of 2 for additional fault tolerance.

```

The exact default storage level and availability/naming of some storage-level constants can depend on the Spark/PySpark version. The core interview distinction is: **cache = default persistence; persist = selectable storage level.**

9. How do you handle missing and duplicate data in a PySpark DataFrame?

`dropna()`

```
df.dropna()
```

Removes rows containing missing/null values.

```

Before:
Alice    50000
Bob      NULL
Charlie  60000

After dropna():
Alice    50000
Charlie  60000

```

fillna()

```
df.fillna({
  "salary": 0
})
```

Replaces missing/null values instead of removing the row.

dropDuplicates()

```
df.dropDuplicates()

df.dropDuplicates(["customer_id"])
```

Removes duplicate rows, either across all columns or based on selected columns.

Missing data:
dropna() → remove
fillna() → replace

Duplicate data:
dropDuplicates() → remove duplicates

10. What is Bucketing in PySpark? How can bucketing improve join performance?

Bucketing divides data into a fixed number of buckets based on one or more bucket columns and stores the data as a table.

```
df.write.mode('overwrite') \
  .bucketBy(4, 'id') \
  .saveAsTable("bucket_table1")

df.write.mode('overwrite') \
  .bucketBy(3, 'dept') \
  .saveAsTable("bucket_table2")
```

The first table is bucketed by id into four buckets. The second is bucketed by dept into three buckets.

Why bucket data?

Without useful bucketing:

Table A ■■■■■■■■
 ■■■ Shuffle → Join
Table B ■■■■■■■■

With compatible bucketing:

Table A
↓
Buckets by join key
↓
Join
↑
Buckets by join key
↑
Table B

When tables are appropriately bucketed on the same join key and other conditions are suitable, Spark may reduce or avoid some shuffle work during joins.

Important observation about your original example

bucket_table1 → bucketed by id
bucket_table2 → bucketed by dept

Join:
b1.id = b2.id

The second table is bucketed by dept, not id. Therefore, this is not a good example for demonstrating a compatible bucketed id join.

Better example for an id join

```
df1.write.mode('overwrite') \
  .bucketBy(4, 'id') \
  .saveAsTable("bucket_table1")

df2.write.mode('overwrite') \
```

```

        .bucketBy(4, 'id') \
        .saveAsTable("bucket_table2")

spark.sql("""
SELECT *
FROM bucket_table1 b1
JOIN bucket_table2 b2
    ON b1.id = b2.id
""").show()

```

Partitioning vs Bucketing

Partitioning
 Data
 ↓
 Partitioned based on column values
 ↓
 Often useful for filtering/pruning

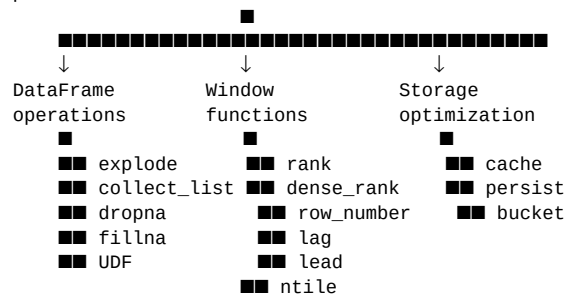
Bucketing
 Data
 ↓
 Fixed number of buckets
 ↓
 Hash-based distribution using bucket column(s)
 ↓
 Can help repeated joins/aggregations on the key

Assignment 9 — Quick Interview Revision

Concept	Remember
UDF	Custom Python function registered for DataFrame column processing
flatten_df()	Normal Python helper; not a UDF
dense_rank()	Same rank for ties, no gaps
rank()	Same rank for ties, gaps possible
row_number()	Unique sequential number
lag()	Previous row
lead()	Next row
ntile(n)	Divide ordered rows into approximately n buckets
explode()	Array → multiple rows
collect_list()	Multiple rows → array
multiLine=True	JSON document/record may span multiple lines
API → PySpark	API → requests → JSON → Spark DataFrame
rowTag	XML element treated as a DataFrame row
Driver	Coordinates the Spark application
Executor	Executes tasks and can cache data
Cluster Manager	Allocates cluster resources
cache()	Persist using default storage level
persist()	Choose storage level
dropna()	Remove rows containing nulls
fillna()	Replace nulls
dropDuplicates()	Remove duplicate rows
Bucketing	Fixed buckets based on bucket columns

Most Important Mental Picture

PySpark



Interview lesson: Before writing code, identify the data pattern. Then choose the PySpark operation that matches the pattern.